

Distributed Systems - 5_11

Today we'll implement the $(1, N)$ atomic register using message passing.

We'll need some assumptions on the capabilities of detecting failures, or the number of correct processes (needs to be above a certain threshold).

We'll start with the Read-Impose Write-All.

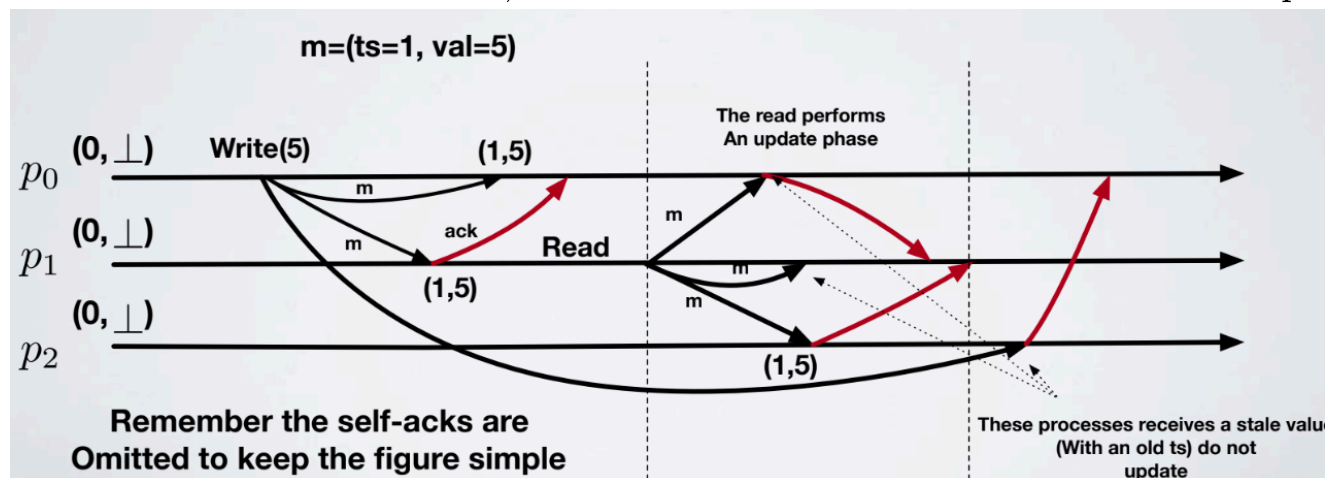
It uses P , beb and p_2p .

What is the idea of this algorithm? It's the same as the one seen with the register. With the register, we wrote the value we read on our row. Before terminating we had to be sure we wrote our value on our row.

So, in message passing, we send our value to everyone, and we wait for an ACK. When we receive ACKs, from everyone who's correct, we can terminate.

What is the structure?

When we want to write a value, we send that value and the associated timestamp.



I terminate my write when I received the ACK from everyone in the system.

If a guy start reading (like where "read" is written), we can't just terminate there, as if p_2 reads after he would read $\perp$ and not 5 as p_1 .

We have to fix the problem that a simple local read will not fix the value for others. So, we do something similar to registers. When we read, we start reading the local value, and send this to everyone. Processes receiving this value will update their local value if $ts_{rec} > ts_{saved}$. The read will terminate when the reading process has received ACKs.

It's called read-impose as the reader tries to impose his value on other readers.

Let's see the code:

```

upon event  $\langle onar, Init \rangle$  do
     $(ts, val) := (0, \perp);$ 
     $correct := \Pi;$ 
     $writeset := \emptyset;$ 
     $readval := \perp;$ 
     $reading := FALSE;$ 

upon event  $\langle \mathcal{P}, Crash \mid p \rangle$  do
     $correct := correct \setminus \{p\};$ 

upon event  $\langle onar, Read \rangle$  do
     $reading := TRUE;$ 
     $readval := val;$ 
    trigger  $\langle beb, Broadcast \mid [WRITE, ts, val] \rangle;$ 

upon event  $\langle onar, Write \mid v \rangle$  do
    trigger  $\langle beb, Broadcast \mid [WRITE, ts + 1, v] \rangle;$ 

upon event  $\langle beb, Deliver \mid p, [WRITE, ts', v'] \rangle$  do
    if  $ts' > ts$  then
         $(ts, val) := (ts', v');$ 
        trigger  $\langle pl, Send \mid p, [ACK] \rangle;$ 

upon event  $\langle pl, Deliver \mid p, [ACK] \rangle$  then
     $writeset := writeset \cup \{p\};$ 

upon  $correct \subseteq writeset$  do
     $writeset := \emptyset;$ 
    if  $reading = TRUE$  then
         $reading := FALSE;$ 
        trigger  $\langle onar, ReadReturn \mid readval \rangle;$ 
    else
        trigger  $\langle onar, WriteReturn \rangle;$ 

```

The writeset contains the ACKs. In readval we save the local value we want to read.

reading=False is used to distinguish the case in which we are the original writer, from the case we are the reader imposing the value.

In the Read Return, we return readval and not val, so we return the value we were imposing on other people.

How many messages for a write? $2N$, N for write, N for ACKs.

A read? $2N$, similar to before.

The step complexity of a write? 2. Same for the read.

Why are we actually using readval in Read Return, and not val? It's possible that val is newer than readval.

Let's take a look at the proof:

- Termination: same for $(1, N)$ regular register
- Validity: same as $(1, N)$ regular register
- Ordering: to complete a read operation, the reader process has to be sure that every other process has in its local copy of the register a value with timestamp bigger or equal of the timestamp of the value read. In this way, any successive read could not return an older value.

Now, can we extend this idea in a Fail-Silent system?

Yes, we can use the quorums. When we write, we'll wait for ACKs from a quorum (majority of ACKs). When I read, it has to read from a quorum.

This pattern, however, is not enough to solve the ordering between reads.

Let us imagine that the system $(p_0, p_1, p_2, p_3, p_4)$ works by having a writer starting $w(5)$, and he dies in the middle. The message for the write will reach only one process, and so p_1 will go to $(1, 5)$. Then p_1 will read and impose the value on everyone, waiting for $3 = \frac{n}{2} + 1$ acks. If p_3 wants to read now, he could receive from itself, p_2 and p_4 the ACKs, and he would read $(0, \perp)$. *Apparently, until now, we weren't imposing the value.*

So, the read will work by also imposing the value he is reading, like the algorithm above. *Important note: the read does not happen locally like in the algorithm above, but a process has to ask the value to other processes, and waits for a quorum.*

So a read will have a first phase, query phase, asking for their values. When he receives a quorum of value, I take the maximum value, and I impose it to others (and wait for ACKs), so this is the imposing phase.

Here is the code:

```
upon event  $\langle onar, Init \rangle$  do
   $(ts, val) := (0, \perp)$ ;
   $wts := 0$ ;
   $acks := 0$ ;
   $rid := 0$ ;
   $readlist := [\perp]^N$ ;
   $readval := \perp$ ;
   $reading := \text{FALSE}$ ;

upon event  $\langle onar, Read \rangle$  do
   $rid := rid + 1$ ;
   $acks := 0$ ;
   $readlist := [\perp]^N$ ;
   $reading := \text{TRUE}$ ;
  trigger  $\langle beb, Broadcast \mid [READ, rid] \rangle$ ;

upon event  $\langle beb, Deliver \mid p, [READ, r] \rangle$  do
  trigger  $\langle pl, Send \mid p, [VALUE, r, ts, val] \rangle$ ;

upon event  $\langle pl, Deliver \mid q, [VALUE, r, ts', v'] \rangle$  such that  $r = rid$  do
   $readlist[q] := (ts', v')$ ;
  if  $\#(readlist) > N/2$  then
     $(maxts, readval) := \text{highest}(readlist)$ ;
     $readlist := [\perp]^N$ ;
    trigger  $\langle beb, Broadcast \mid [WRITE, rid, maxts, readval] \rangle$ ;
```

```

upon event  $\langle onar, Write \mid v \rangle$  do
     $rid := rid + 1;$ 
     $wts := wts + 1;$ 
     $acks := 0;$ 
    trigger  $\langle beb, Broadcast \mid [WRITE, rid, wts, v] \rangle;$ 

upon event  $\langle beb, Deliver \mid p, [WRITE, r, ts', v'] \rangle$  do
    if  $ts' > ts$  then
         $(ts, val) := (ts', v');$ 
    trigger  $\langle pl, Send \mid p, [ACK, r] \rangle;$ 

upon event  $\langle pl, Deliver \mid q, [ACK, r] \rangle$  such that  $r = rid$  do
     $acks := acks + 1;$ 
    if  $acks > N/2$  then
         $acks := 0;$ 
        if  $reading = TRUE$  then
             $reading := FALSE;$ 
            trigger  $\langle onar, ReadReturn \mid readval \rangle;$ 
        else
            trigger  $\langle onar, WriteReturn \rangle;$ 

```

When we have an algorithm with quorums, we have to distinguish requests, so we use a rid for the requests coming from a process.

When we are a writer and we write, we also increase the rid by 1. This as we have to distinguish the ACKs for a specific operation of write.

How many messages we consume if we want to write something? $2N$. A read? $4N$, with $2N$ for the first phase (the query), and $2N$ for the imposing phase.

The step complexity of a write? 2. For a Read? 4.

The proof:

- Termination and Validity: same for $(1, N)$ regular register
- Ordering: due to the fact that the read imposes the write of the value read to a majority of processes and to the property of intersection of quorums.

Let's do an exercise:

Exercise 4.6: Explain why the request identifier included in WRITE, ACK, READ, and VALUE messages of the “Majority Voting” algorithm (Algorithm 4.2) can be omitted in the “Read-One Write-All” algorithm (Algorithm 4.1).

Read-one Write-all Algorithm

```

upon event  $\langle onrr, Init \rangle$  do
     $val := \perp$ ;
     $correct := \Pi$ ;
     $writeset := \emptyset$ ;

upon event  $\langle \mathcal{P}, Crash \mid p \rangle$  do
     $correct := correct \setminus \{p\}$ ;

upon event  $\langle onrr, Read \rangle$  do
    trigger  $\langle onrr, ReadReturn \mid val \rangle$ ;

upon event  $\langle onrr, Write \mid v \rangle$  do
    trigger  $\langle beb, Broadcast \mid [WRITE, v] \rangle$ ;

upon event  $\langle beb, Deliver \mid q, [WRITE, v] \rangle$  do
     $val := v$ ;
    trigger  $\langle pl, Send \mid q, ACK \rangle$ ;

upon event  $\langle pl, Deliver \mid p, ACK \rangle$  do
     $writeset := writeset \cup \{p\}$ ;

upon  $correct \subseteq writeset$  do
     $writeset := \emptyset$ ;
    trigger  $\langle onrr, WriteReturn \rangle$ ;

```

Read-one Write-all Algorithm

```

upon event  $\langle onrr, Init \rangle$  do
     $(ts, val) := (0, \perp)$ ;
     $wts := 0$ ;
     $acks := 0$ ;
     $rid := 0$ ;
     $readlist := [\perp]^N$ ;

upon event  $\langle onrr, Write \mid v \rangle$  do
     $wts := wts + 1$ ;
     $acks := 0$ ;
    trigger  $\langle beb, Broadcast \mid [WRITE, wts, v] \rangle$ ;

upon event  $\langle beb, Deliver \mid p, [WRITE, ts', v'] \rangle$  do
    if  $ts' > ts$  then
         $(ts, val) := (ts', v')$ ;
    trigger  $\langle pl, Send \mid p, [ACK, ts'] \rangle$ ;

upon event  $\langle pl, Deliver \mid q, [ACK, ts'] \rangle$  such that  $ts' = wts$  do
     $acks := acks + 1$ ;
    if  $acks > N/2$  then
         $acks := 0$ ;
        trigger  $\langle onrr, WriteReturn \rangle$ ;

upon event  $\langle onrr, Read \rangle$  do
     $rid := rid + 1$ ;
     $readlist := [\perp]^N$ ;
    trigger  $\langle beb, Broadcast \mid [READ, rid] \rangle$ ;

upon event  $\langle beb, Deliver \mid p, [READ, r] \rangle$  do
    trigger  $\langle pl, Send \mid p, [VALUE, r, ts, val] \rangle$ ;

upon event  $\langle pl, Deliver \mid q, [VALUE, r, ts', v'] \rangle$  such that  $r = rid$  do
     $readlist[q] := (ts', v')$ ;
    if  $\#(readlist) > N/2$  then
         $v := highestval(readlist)$ ;
         $readlist := [\perp]^N$ ;
        trigger  $\langle onrr, ReadReturn \mid v \rangle$ ;

```

In the first algorithm, we do not use a timestamp on the write, and so there is no ID.

The write is completed when we receive an ACK from everyone. Is it possible that once I complete a w_1 operation and start w_2 , is it possible that there are pending ACKs for w_1 ? No.

In the second algorithm, a write terminates when he sees the majority. It's possible that, when we start w_2 , I did not receive all ACKs for w_1 , and so we need to identify that is an ACK for w_1 and not w_2 .

What happens in the second algorithm if I remove the condition $r = \text{rid}$ in the last handler?

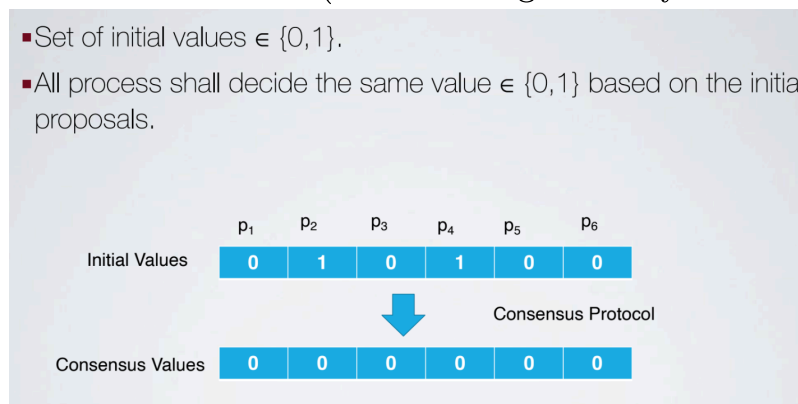
See if we can create a counterexample. Spoiler: The algorithm breaks.

Synchronous Consensus

Once we can solve consensus, we can solve **almost** all problems that matters.

We'll see how to solve the consensus problem in synchronous and eventually synchronous setting, for which we'll see the Paxos algorithm.

In databases, we have the consensus problem when we have to decide if a transaction has to commit or abort (like inserting an entry in a table).



At a certain point we have to decide, we can't have p_1 saying to abort and p_2 to proceed. And so the consensus problem is the problem where everyone has its own opinion, and then we have to decide if we have to abort for everyone or commit for everyone. So any process can propose a value (0,1), and then everyone has to decide a value that is the same for everyone. This kind of consensus is known as Binary Consensus.

What is the specification of Consensus?

In the consensus primitive we have two actions:

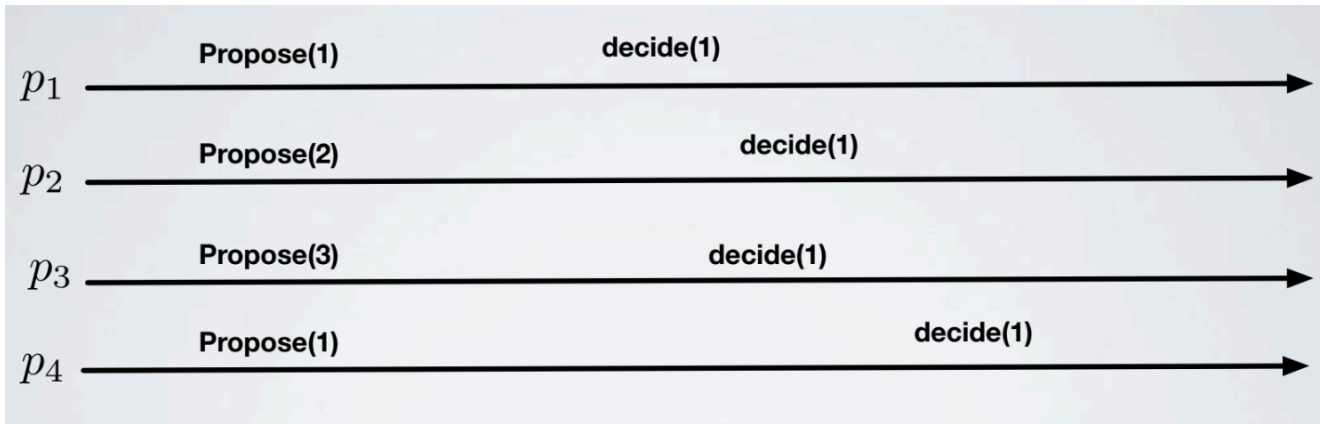
- Propose | v , proposing a value v ,
- Decide | v , outputs a decided value v of consensus

We have the following properties:

- (Termination): Every correct process eventually decides some value
- (Validity): if a process decides v , then v was proposed by some process
- (Integrity): No process decides twice

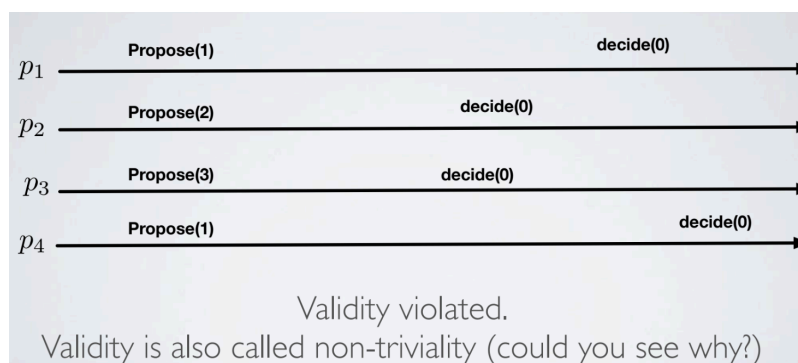
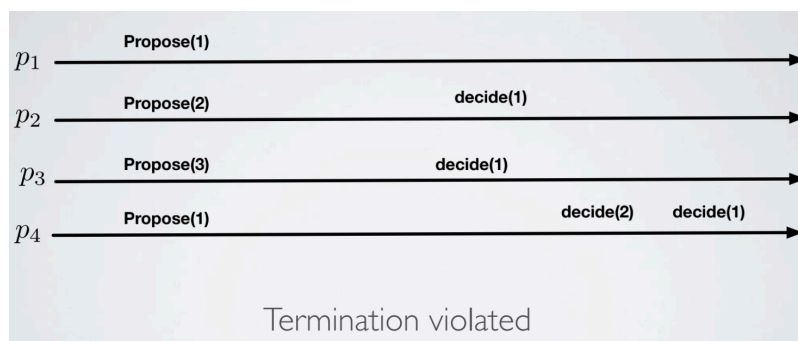
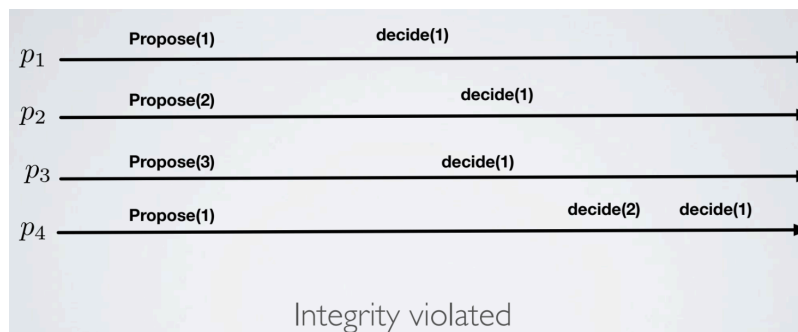
- (Agreement): No two correct processes decide differently.

Let's see an execution:

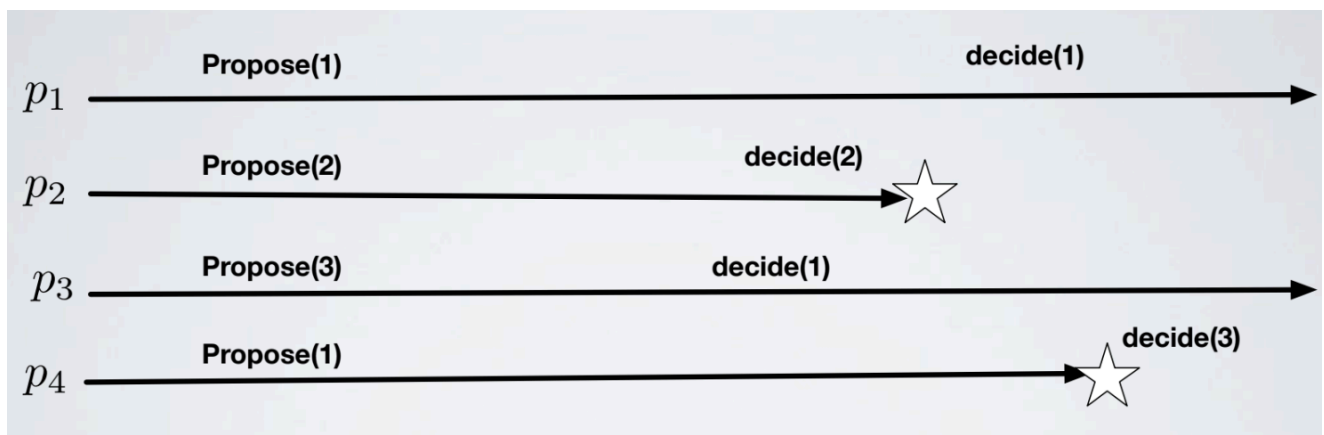


Is this correct? Yes, every property is valid.

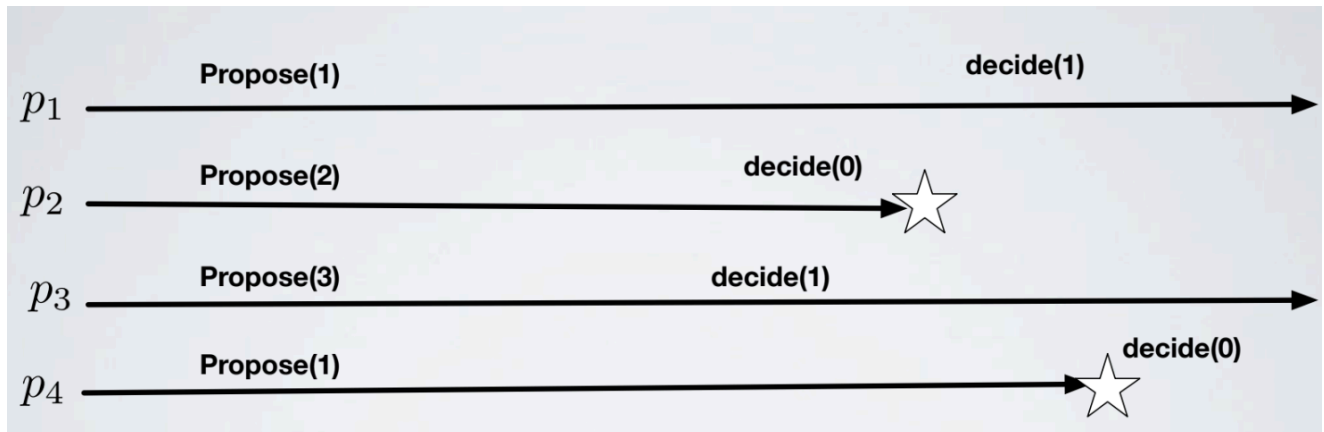
Note: the consensus is not made on the most frequent value.



Why without validity, we can create an algorithm that is trivial? We can create an algorithm, for example, that always decides 0, but this obviously has no sense. Is this execution ok?



Yes.



Here it's not correct as we are violating validity, p_2 is deciding 0 even though it has not been proposed by anyone, and this violates the fact even though p_2 is not correct (check the definition of validity, it does not specify "correct").